Module6

Exercise4 (Auth & Error handling)

Q1.) User sign up with email and password

A1.) Code

```
router.post("/signup", async (req, res) => {
  const { email, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);

  try {
    const result = await pool.query(
      "INSERT INTO users (email, password) VALUES ($1, $2) RETURNING *",
      [email, hashedPassword]
    );

    res.status(201).json(result.rows[0]);
  } catch (err) {
    if (err.code === "23505") {
      return res.status(400).json({ message: "User already exists" });
    }
    res.status(500).json({ error: err.message });
  }
});
```

Output

POST http://localhost:8000/user/signup

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2     "email":"dhruv@gmail.com",
3     "password":"Dhruvpa"
4 }
```

Body Cookies Headers (7) Test Results

Pretty Raw Preview JSON


```
1 {"id":1,"email":"dhruv@gmail.com","password":"$2b$10$5Vdkbou3qlaZRMRQtOyT1ey1T7hVa5jX3IrIDn44R/6OOWUpX6IU."}
```

Q2.) User login and return token after validating password and return token

A2.) Code

```
router.post("/login", async (req, res) => {
  const { email, password } = req.body;

  const result = await pool.query(
    "SELECT * FROM users WHERE email = $1",
    [email]
  );

  const user = result.rows[0];

  if (!user) {
    return res.status(400).json({ message: "Invalid credentials" });
  }

  const isMatch = await bcrypt.compare(password, user.password);

  if (!isMatch) {
    return res.status(400).json({ message: "Invalid credentials" });
  }

  const token = jwt.sign(
    { id: user.id, email: user.email },
    "SECRET_KEY",
    { expiresIn: "1h" }
  );

  res.json({ token });
});
```

Output

http://localhost:8000/user/login

POST http://localhost:8000/user/login

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2     "email":"dhruv@gmail.com",
3     "password":"Dhruvpa"
4 }
```

Body Cookies Headers (7) Test Results

Pretty Raw Preview JSON


```
1 {"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOi
```

Q3.) Apply auth Middleware to all routes (timestamp not valid, token expire , token malformed.

A3.) Code

```
Module6 > Exercise4 > middleware.js > authMiddleware
const jwt = require("jsonwebtoken");

const authMiddleware = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Token missing" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, "SECRET_KEY");
    req.user = decoded;

    next();

  } catch (err) {
    if (err.name === "TokenExpiredError") {
      return res.status(401).json({ message: "Token expired" });
    }

    if (err.name === "JsonWebTokenError") {
      return res.status(401).json({ message: "Malformed token" });
    }

    return res.status(401).json({ message: "Invalid token" });
  }
};

module.exports = authMiddleware;
```

Output

http://localhost:8000/user/profile

GET http://localhost:8000/user/profile

Params Authorization Headers (10) Body Scripts Settings

Headers 9 hidden

	Key	Value
☑	Authorization	Bearer eyJhbGciOiJIUzI1N
	Key	Value

Body Cookies Headers (7) Test Results

Pretty Raw Preview JSON


```
1 {"user":{"id":1,"email":"dhruv@gmail.com","iat":1773836002,"exp":1773839602}}
```